

TEMPERATURE MONITORING NEURAL NETWORK USING ReLU

Project Title: Temperature Monitoring Neural Network Using ReLU

Application: Temperature Monitoring

Concept: ReLU Activation Function

Dataset: Temperature Dataset

1. Problem Statement

A neural network for temperature monitoring frequently receives negative pre-activation values. Implement a simple neuron using ReLU and test it with positive and negative temperature-related input values. Analyze the resulting activation outputs, compare the responses for positive and negative inputs, and determine how ReLU affects the representation of the input values.

2. Objective

- Implement a simple artificial neuron for temperature inputs.
- Calculate the pre-activation value using weight and bias.
- Apply the ReLU activation function.
- Test the neuron using positive and negative temperature values.
- Compare negative and positive pre-activation responses.
- Analyze how ReLU affects the representation of input values.
- Generate visualizations of the activation behavior.

3. Concept Used

The artificial neuron uses the linear transformation: $z = w \times x + b$. Where: x = temperature input; $w = 0.5$; $b = -8$. The pre-activation value is: $z = 0.5x - 8$. The ReLU activation function is: $\text{ReLU}(z) = \max(0, z)$. ReLU converts negative values to zero, passes positive values unchanged, introduces non-linearity, and produces sparse activations.

3.1 ReLU Activation Function

Property	Description
Negative pre-activation	Converted to zero.
Positive pre-activation	Passed through unchanged.
Zero pre-activation	Remains zero.
Non-linearity	Produces a piecewise linear response.
Sparsity	Produces zero activation for negative values.

4. Methodology

Input: Load temperature values from dataset/temperature_data.csv.

Neuron parameters: Set weight $w = 0.5$ and bias $b = -8$.

Weighted input: Calculate $w * x$.
Pre-activation: Calculate $z = wx + b$.
ReLU activation: Calculate $\text{ReLU}(z) = \max(0,z)$.
Analysis: Compare the pre-activation values with the ReLU outputs.
Sparsity: Count zero and non-zero activation outputs.
Visualization: Generate graphs showing temperature, pre-activation, and ReLU output.
Output: Save the generated visualizations and execution screenshot.

5. Implementation

5.1 Tools & Libraries

Tool / Library	Use
Python	Core programming language
NumPy	Numerical calculations and ReLU operation
Pandas	Loading and processing CSV data
Matplotlib	Visualization and plotting
Jupyter Notebook	Interactive step-by-step analysis

5.2 Project Structure

None

```
temperature-relu-neuron/  
├── README.md  
├── requirements.txt  
├── dataset/  
│   └── temperature_data.csv  
├── src/  
│   └── relu_temperature_neuron.py  
├── notebooks/  
│   └── ReLU_Temperature_Analysis.ipynb  
├── results/  
│   ├── relu_activation_plot.png  
│   └── temperature_activation_comparison.png  
├── screenshots/  
│   └── output.png
```

5.3 Source Code / Github Repository

URL: <https://github.com/HariM917/temperature-relu-neuron>

The complete source code, dataset, Jupyter notebook, generated results, README documentation, and execution screenshot are maintained in the GitHub repository.

6. Results & Output

The program implements a single temperature-monitoring neuron using a weight of 0.5 and a bias of -8. The pre-activation is calculated using $z = wx + b$, followed by ReLU activation.

Temp (°C)	Weighted Input	Bias	Pre-activation (z)	ReLU Output
-20	-10.0	-8	-18.0	0.0
-10	-5.0	-8	-13.0	0.0
0	0.0	-8	-8.0	0.0
5	2.5	-8	-5.5	0.0
10	5.0	-8	-3.0	0.0
20	10.0	-8	2.0	2.0
30	15.0	-8	7.0	7.0
40	20.0	-8	12.0	12.0

Output Summary

- Total inputs: 8
- Zero outputs: 5
- Non-zero outputs: 3
- Sparsity: 62.5%

Example calculations:

For -20°C: $z = (0.5 \times -20) - 8 = -18$. $\text{ReLU}(-18) = 0$.

For 40°C: $z = (0.5 \times 40) - 8 = 12$. $\text{ReLU}(12) = 12$.

Threshold: $0.5x - 8 = 0$, therefore $x = 16^\circ\text{C}$.

FIGURES

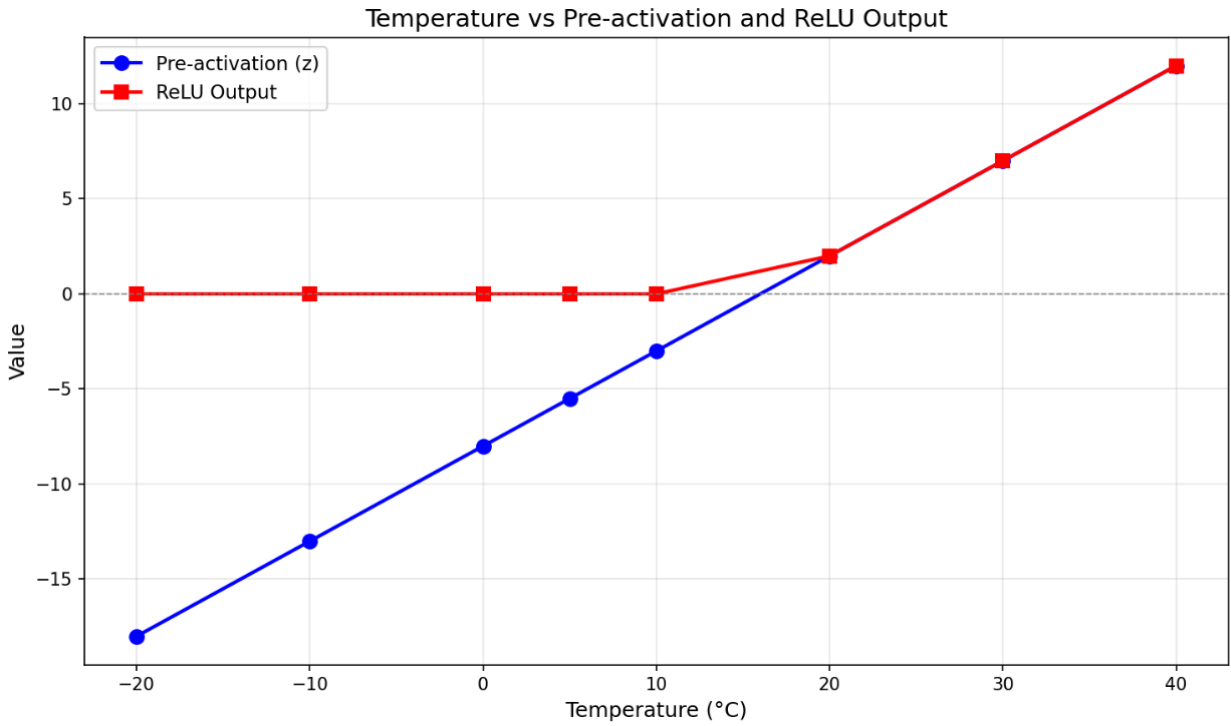


Figure 1: Temperature versus pre-activation and ReLU output.

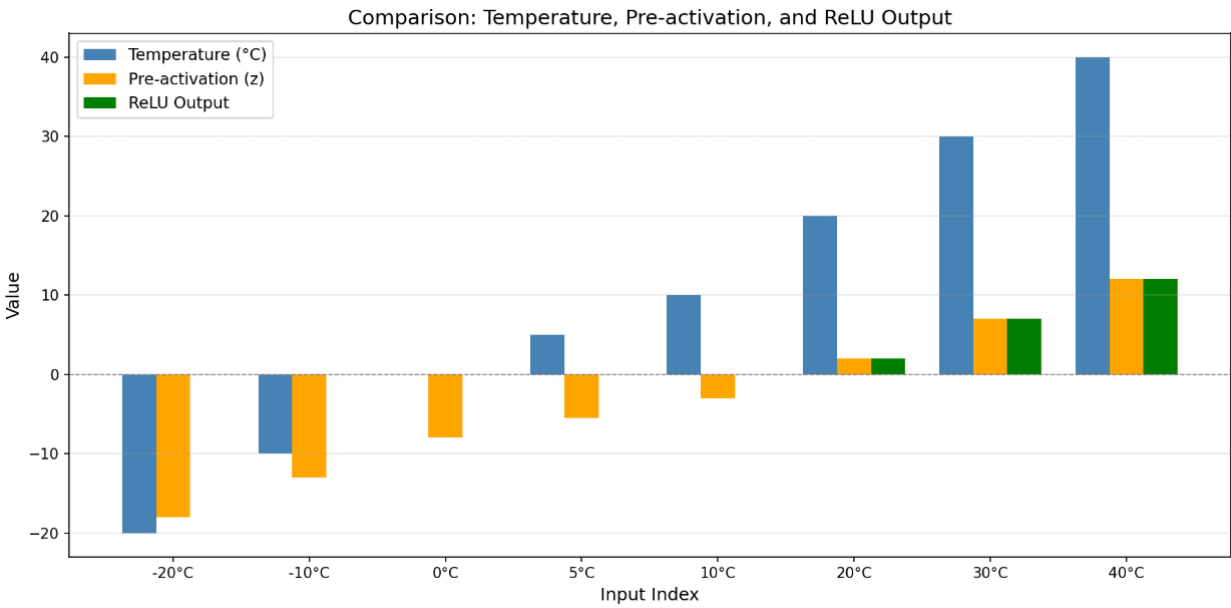


Figure 2: Comparison of temperature, pre-activation, and ReLU output.

```

Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\HARI> cd "C:\Users\HARI\OneDrive\Desktop\DLE IA\temperature-relu-neuron"
PS C:\Users\HARI\OneDrive\Desktop\DLE IA\temperature-relu-neuron> python src\relu_temperature_neuron.py
Loaded 8 temperature values: [-20 -10 0 5 10 20 30 40]

=====
TEMPERATURE NEURON ACTIVATION RESULTS
=====

Neuron Parameters:
Weight = 0.5
Bias = -8

Formula:
z = 0.5 * temperature + (-8)

Activation:
ReLU(z) = max(0, z)

-----
Temperature (°C) | Weighted Input | Bias | Pre-activation | ReLU Output
-----
-20.0 | -10.0 | -8.0 | -18.0 | 0.0
-10.0 | -5.0 | -8.0 | -13.0 | 0.0
0.0 | 0.0 | -8.0 | -8.0 | 0.0
5.0 | 2.5 | -8.0 | -5.5 | 0.0
10.0 | 5.0 | -8.0 | -3.0 | 0.0
20.0 | 10.0 | -8.0 | 2.0 | 2.0
30.0 | 15.0 | -8.0 | 7.0 | 7.0
40.0 | 20.0 | -8.0 | 12.0 | 12.0
-----

Summary:
Total inputs: 8
Zero outputs: 5 (neuron did NOT fire)
Non-zero outputs: 3 (neuron fired)
Sparsity: 5/8 = 62.5%

=====

Plot saved: C:\Users\HARI\OneDrive\Desktop\DLE IA\temperature-relu-neuron\results\relu_activation_plot.png
Plot saved: C:\Users\HARI\OneDrive\Desktop\DLE IA\temperature-relu-neuron\results\temperature_activation_comparison.png

Done! Check the 'results/' folder for saved plots.
PS C:\Users\HARI\OneDrive\Desktop\DLE IA\temperature-relu-neuron> |

```

Figure 3: Program execution showing neuron activation results and summary analysis.

7. Analysis

- The experiment demonstrates that ReLU suppresses negative pre-activation values by converting them to zero.
- For temperatures from -20°C to 10°C, the calculated pre-activation values are negative and therefore produce zero ReLU outputs.
- For temperatures 20°C, 30°C, and 40°C, the pre-activation values are positive and ReLU passes them unchanged.
- The threshold for the selected weight and bias is 16°C.
- Five out of eight inputs produce zero activation.
- The resulting sparsity is 62.5%.
- The experiment demonstrates that ReLU changes the representation of the pre-activation values by suppressing negative responses while preserving positive responses.

8. Conclusion

This mini project demonstrates the use of the ReLU activation function in a simple temperature-monitoring neuron. The neuron calculates the pre-activation using $z = wx + b$ and applies ReLU to the result. Negative pre-activation values are converted to zero, while positive values are passed unchanged.

For the selected dataset, five of eight inputs produce zero outputs, resulting in 62.5% sparsity. The project demonstrates the thresholding, non-linearity, and sparse activation behavior of ReLU.

9. References

1. Nair, V., & Hinton, G. E. (2010). Rectified Linear Units Improve Restricted Boltzmann Machines. ICML-10.
2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
3. Glorot, X., Bordes, A., & Bengio, Y. (2011). Deep Sparse Rectifier Neural Networks. AISTATS.
4. NumPy Documentation - <https://numpy.org/doc/>
5. Pandas Documentation - <https://pandas.pydata.org/docs/>
6. Matplotlib Documentation - <https://matplotlib.org/stable/contents.html>
7. Project GitHub Repository - <https://github.com/HariM917/temperature-relu-neuron>